

Unity Physics

Forces and contacts

The two things a physics engine really does: push objects around, and stop them going through each other.

$$F = m \times a$$

$$\text{gravity} = 9.81 \text{ m/s}^2$$



Three components do all the work



Rigidbody

Makes the object obey physics: it falls, it can be pushed, it keeps moving once it is moving. Without a Rigidbody an object is invisible to forces — it just sits wherever you put it.



Collider

The shape the engine uses to work out touching. It does not have to match your model — a box, sphere or capsule is far cheaper and is what most games actually use.



Physic Material

How a surface behaves on contact: how slippery it is, and how much it bounces. Optional — leave it off and you get a middling, slightly grippy default.

The rule that catches everyone out

Two objects only collide if both have a Collider and at least one of them has a Rigidbody. A wall or a floor normally has a Collider and no Rigidbody — the engine treats it as immovable, which is exactly what you want and is also cheaper to simulate.

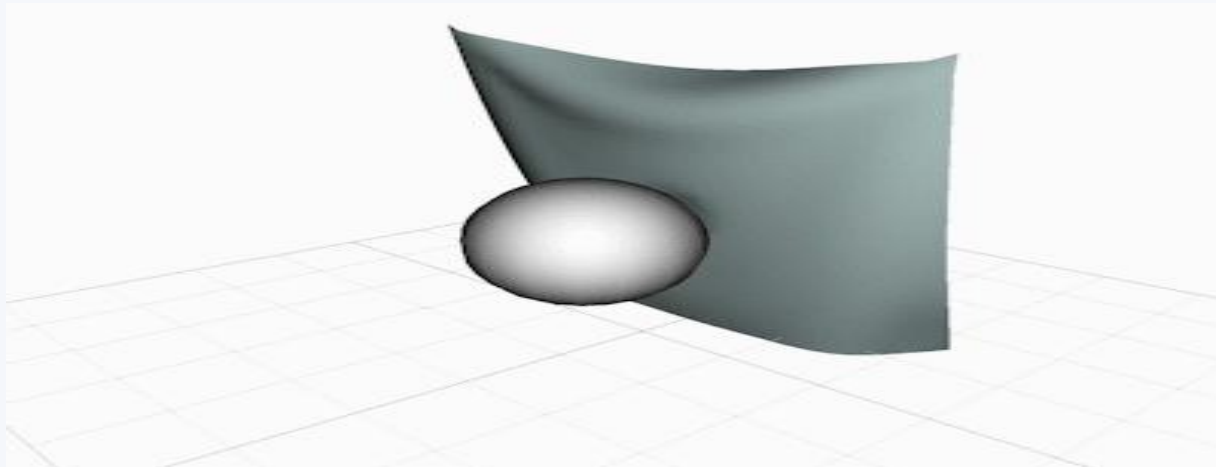
Rigid bodies, and what they are not

Rigid body

A body that never deforms: the distance between any two points on it stays the same however it moves. Crates, cars, coins, most characters. It is the only kind a Rigidbody component simulates.

Soft body

A body whose shape can change, while still holding its form rather than flowing away like a fluid. Cloth, rope and springs — all handled by separate systems in Unity.



What adding a Rigidbody gives you

1

Reacts to forces and gravity

It sets the object's mass and drag, and the momentum it carries into a contact.

2

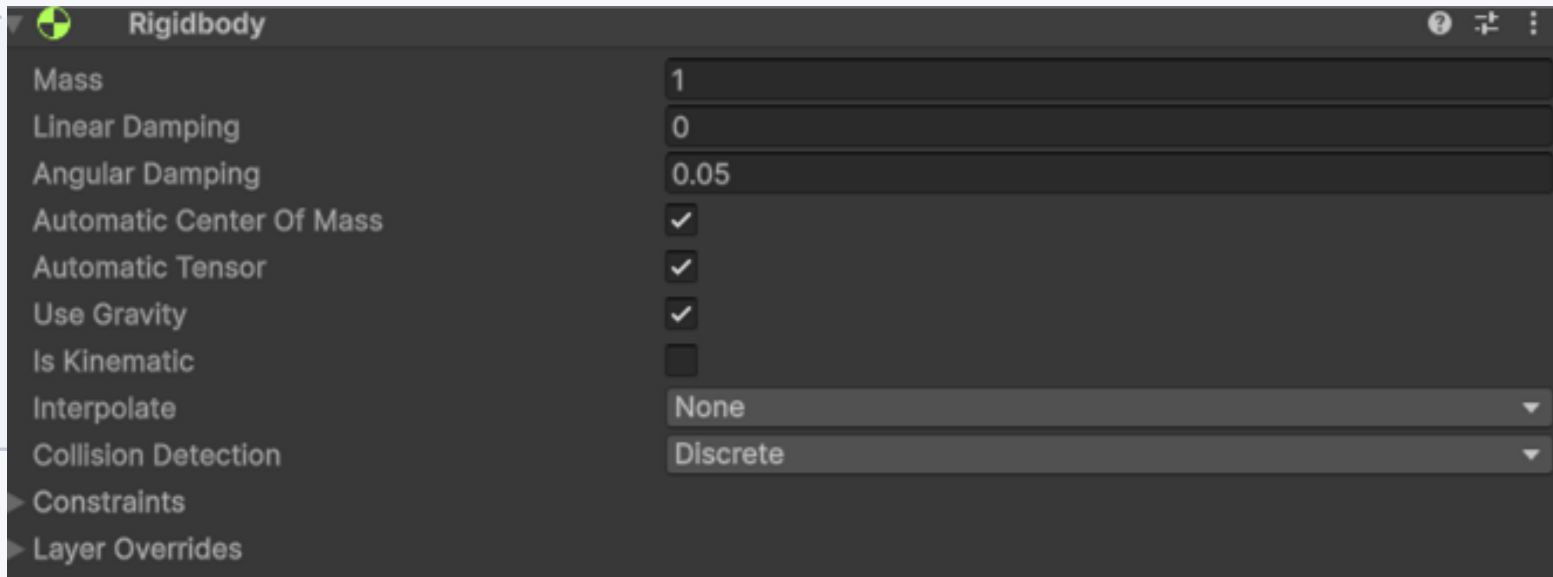
It owns the movement now

Writing to `transform.position` fights it. Apply forces, or use `MovePosition`.

3

Constraints freeze axes

Tick Freeze Position or Freeze Rotation to remove a degree of freedom outright.



Three kinds of Physics object : Static, kinematic and dynamic

	Components	Moved by	How it behaves
Static	Collider only, NO Rigidbody	Nothing — it is meant to stay put and NEVER MOVE.	Ignores every force. The engine assumes it never moves, so moving one in code does not reliably produce collision events. Floors, walls, level geometry.
Kinematic	Rigidbody with Is Kinematic option checked , plus a Collider	Your code by moving the Transform or through other systems like animation or network	Mass, drag and gravity are all ignored, but its collisions still update as it moves. It pushes dynamic objects around and nothing pushes it back. Use for stuff like : Moving platforms, doors,
Dynamic	Rigidbody with Is Kinematic option Unchecked, plus a Collider	The physics engine	Falls, gets pushed, carries momentum. Collides with static, kinematic and other dynamic objects. This is the default

Rule of thumb: a dynamic object needs a Rigidbody, and everything it should notice needs a Collider. If a collision is not happening, check which of the three kinds each object actually is — two static objects will never report anything, however hard they overlap.

Unresolvable scenario When the solver has no answer at all

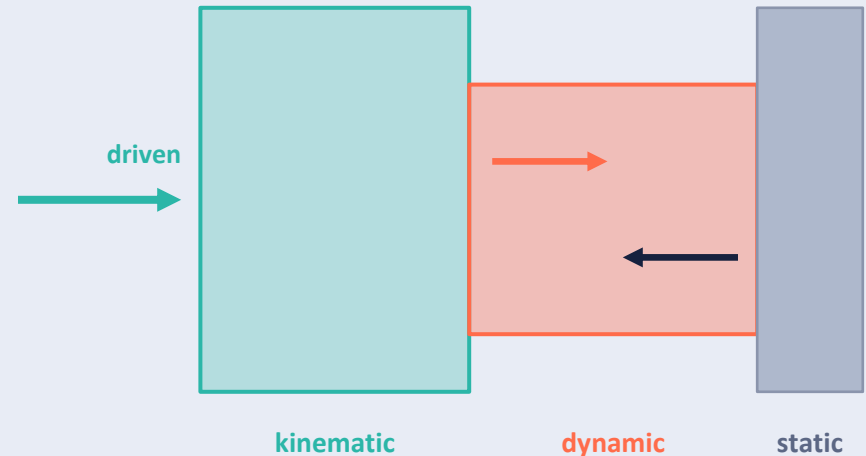
Only one of the three can move

- The static wall is immovable, and the kinematic platform ignores contact impulses entirely — its motion comes from your code, not from physics.
- So the whole response has to land on the crate, and it is being told two contradictory things: get out of the wall, and get out of the platform.
- There is no velocity that satisfies both. The engine is not failing — you have asked it for something that does not exist.

So it picks the least bad option

- The crate buzzes, sinks into one surface, or squirts out sideways.
- If the overlap gets deep enough in one step, it is pushed clean through the wall.

A crate with nowhere to go



no velocity for the crate satisfies both contacts at once

What actually collides with what

Solid collision

They stop each other, and OnCollision... fires

	Static	Kinematic	Dynamic
Static	—	—	Yes
Kinematic	—	—	Yes
Dynamic	Yes	Yes	Yes

Trigger overlap

They pass through, and OnTrigger... fires

	Static	Kinematic	Dynamic
Static	—	Yes	Yes
Kinematic	Yes	Yes	Yes
Dynamic	Yes	Yes	Yes

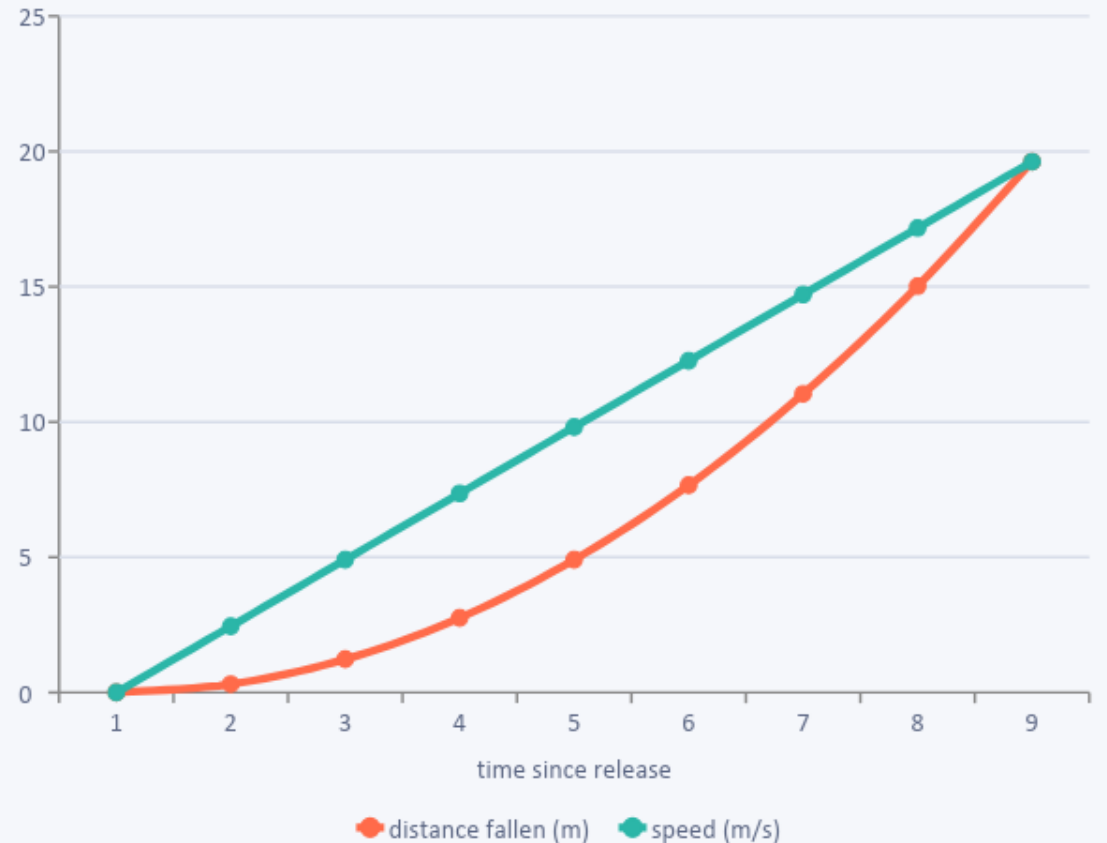
The rule behind both grids: the engine only reports a pair if at least one of the two has a Rigidbody, and it only makes them solid if at least one of them is dynamic. Two static objects are invisible to each other no matter how deeply they overlap — and a kinematic object will slide straight through a wall, because nothing in that pair is being driven by physics.

Gravity treats every object the same

$$d = \frac{1}{2} g t^2 \quad v = g t$$

- Gravity is an **acceleration**, not a force: -9.81 m/s^2 downward, applied to **every** Rigidbody equally.
- So heavy things do not fall faster. A 1 kg crate and a 100 kg crate land together. Mass only matters once objects start pushing each other.
- Rigidbody.drag stops the speed climbing for ever: an object settles at roughly 9.8 divided by the drag value.

A dropped object, second by second



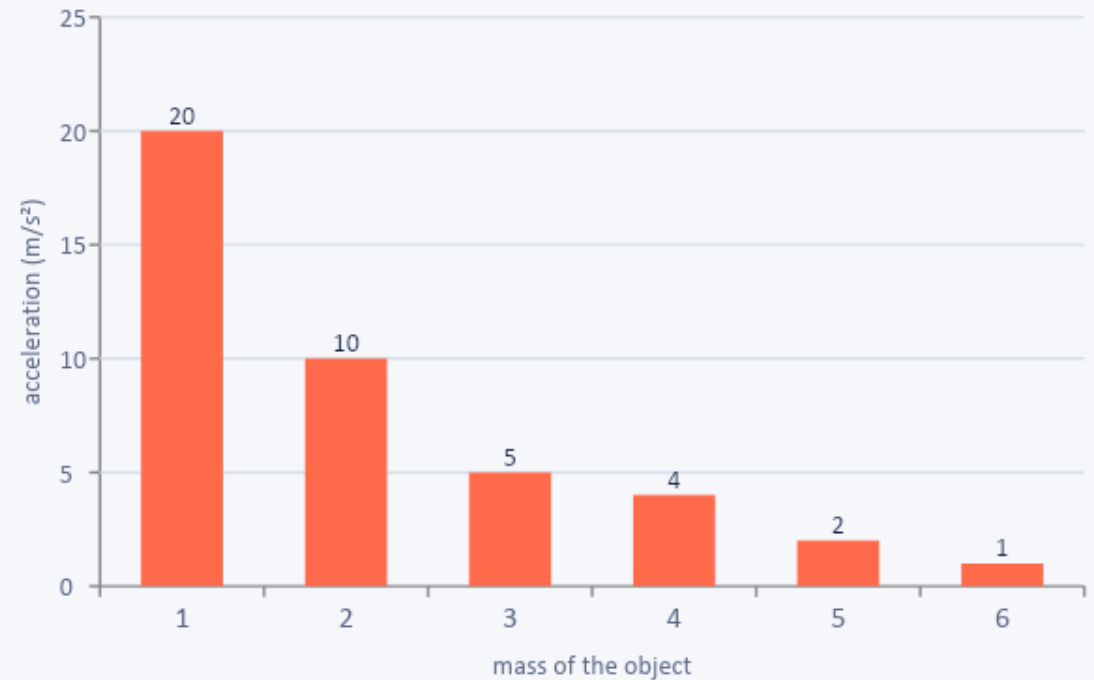
After one second it has fallen about 4.9 m and is moving at 9.8 m/s — a useful pair of numbers to keep in your head.

Force, mass and acceleration

$$F = ma \implies a = \frac{F}{m}$$

- Force is the cause, acceleration is the effect. A force never sets a speed — it sets how fast the speed is changing.
- Mass is the resistance. The same push on twice the mass gives half the acceleration.
- Double the force and you double the acceleration: the two rise together, exactly in step.
- Acceleration is measured in metres per second, per second. 5 m/s^2 means the speed gains 5 m/s for every second the force lasts.
Worked through: 20 N on a 4 kg crate gives 5 m/s^2 . Hold it for two seconds and the crate is moving at 10 m/s — and it keeps moving when you stop.

The same 20 N push on six different objects



One push, six answers. Nothing in $F = ma$ mentions how fast the object is already going — only how quickly that speed will change.

Applying a force in code

Where to write it

```
void FixedUpdate() { rb.AddForce(f); }
```

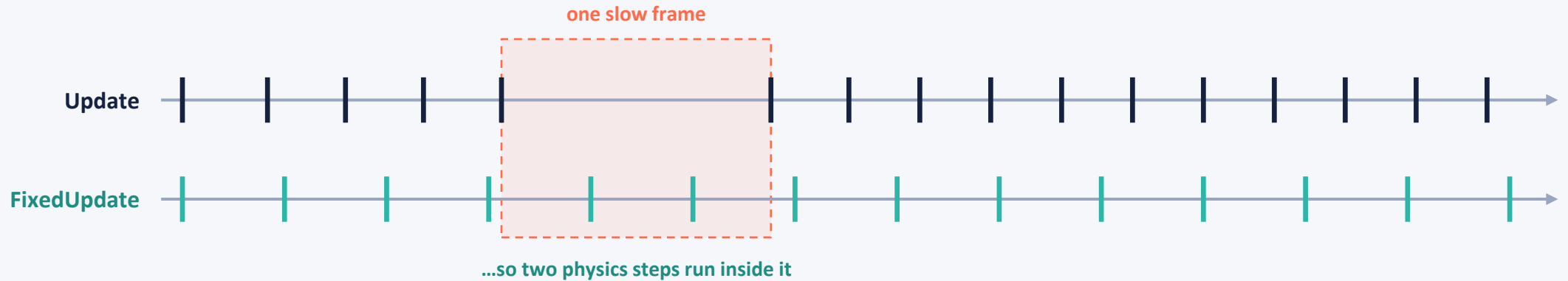
Physics runs on its own clock, 50 steps a second, whatever the frame rate. Apply forces there and every machine behaves the same. The next slide takes that apart.



ForceMode	What you are handing it	Reach for it when...
Force	A steady push, in newtons. Heavier objects respond less.	Thrust, wind, a conveyor — anything sustained.
Acceleration	A steady push that ignores mass entirely.	Custom gravity, or a pull that must feel the same on everything.
Impulse	One instant kick. Heavier objects respond less.	An explosion, a bullet hit, a bat striking a ball.
VelocityChange	One instant kick that ignores mass.	A jump or a dash that must feel identical at any mass.

The two instant modes are for events; the two steady modes are the ones you call every FixedUpdate.

Two clocks: Update and FixedUpdate



Update is whatever the machine manages. FixedUpdate is fifty a second, always — sometimes twice in a frame, sometimes not at all.

Put it in FixedUpdate

- AddForce, and setting velocity directly
- MovePosition and MoveRotation
- Reading a Rigidbody to decide something
- Anything that must behave the same at 30 fps and at 240

Leave it in Update

- Reading input — `GetButtonDown` is true for one frame only, and `FixedUpdate` can miss it entirely
- Camera work, UI, animation, effects
- Anything the player watches rather than feels

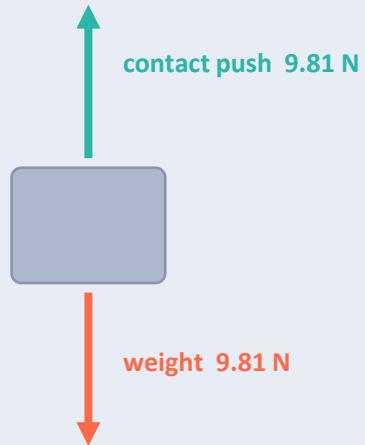
```
bool jump;
void Update() {
    if (Input.GetButtonDown("Jump"))
        jump = true;
}

void FixedUpdate() {
    if (jump) { Jump(); jump = false; }
}
```

Inside FixedUpdate, `Time.deltaTime` already gives you the fixed step. Change it in Project Settings › Time — halving it doubles the cost.

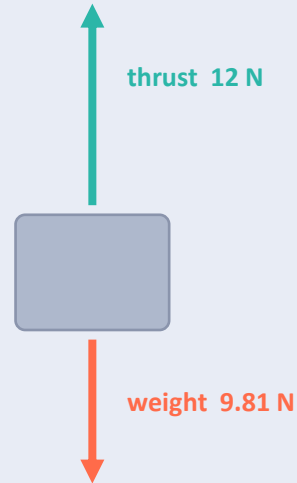
Forces add up. They are vectors.

A crate on the floor



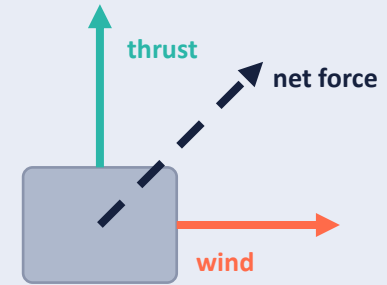
Equal and opposite, so the total is zero — and nothing moves.

A 1 kg rocket lifting off



$12 - 9.81$ leaves 2.19 N upward, so it accelerates at 2.19 m/s^2 .

Thrust plus a side wind



Pointing different ways, they combine into one diagonal push.

Forces are vectors: same direction and they add, opposite and they subtract, at an angle and they resolve into a single diagonal. However many you apply, the engine reduces them to one arrow before anything moves.

Motion comes from the sum of the forces this step

$$\sum \vec{F} = \vec{F}_1 + \vec{F}_2 + \vec{F}_3 + \dots \quad \vec{a} = \frac{\sum \vec{F}}{m}$$

1 AddForce does not move anything

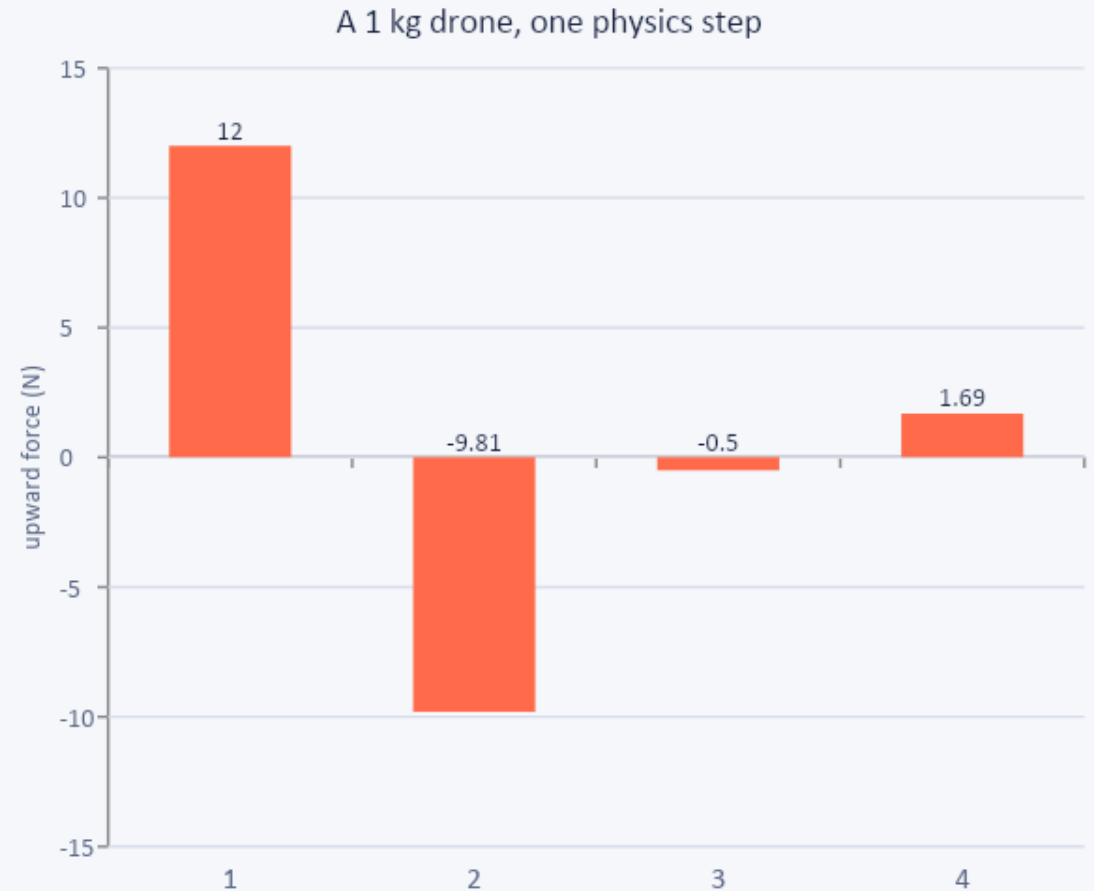
It adds a vector into a running total the Rigidbody keeps for this step. When the step runs, that one total — nothing else — decides how the object moves: $\text{total} \div \text{mass} = \text{this step's acceleration}$.

2 Forces are vectors, so they can cancel

Two equal pushes in opposite directions add up to nothing, and the object sits still — not because the engine ignored you, but because the total came out zero. A crate resting on the floor is exactly this: weight down, contact push up.

3 The total is wiped at the end of the step

Next step starts from zero. A sustained push has to be re-applied every `FixedUpdate`; a single `AddForce` is a one-off nudge lasting one fiftieth of a second, which is why it looks like nothing happened.



The engine never sees three forces — it sees 1.69 N up, divides by the 1 kg mass, and accelerates the drone at 1.69 m/s² for this step only.

How a force turns into movement

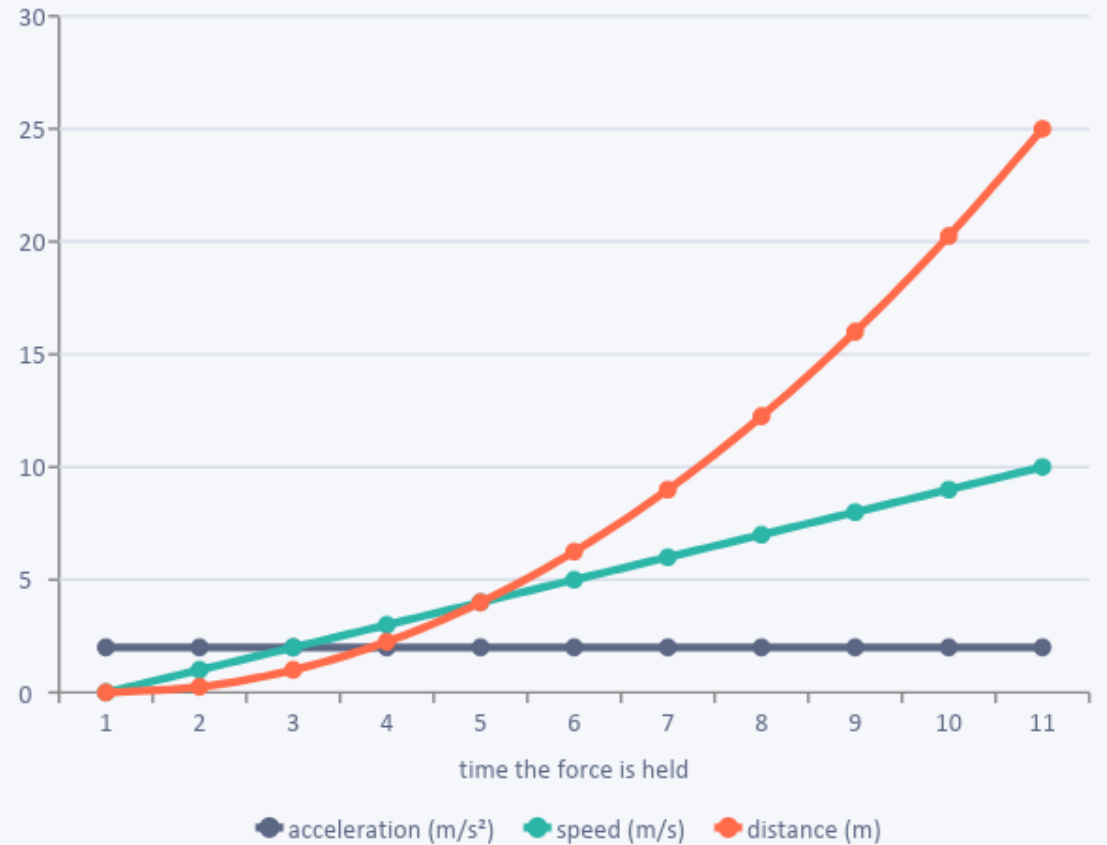
force → acceleration → speed → position

A force never sets an object's position, and it never sets its speed either. It sets how fast the speed is changing, and the speed in turn sets how fast the position is changing. Each link only feeds the next one.

Reading the movement off the forces

- Net force of zero: the speed stops changing. The object keeps drifting at whatever speed it already had.
- Steady net force: the speed climbs steadily and the path curves — exactly what gravity does to a thrown ball.
- Force against the motion: the object slows, turns round, and comes back. It never stops instantly.
- So you shape the forces and let the movement follow. Dragging the position by hand fights the engine instead of using it.

One steady push, held for five seconds



The push never changes, yet the speed rises in a straight line and the distance curves upward — that is the whole of $F = m \times a$ in one picture.

Impulse: a force bundled up with its duration

$$\vec{J} = \vec{F} \Delta t = m \Delta \vec{v}$$

Why it is a separate idea

- Some pushes are far shorter than a physics step — a bat meets a ball for about two thousandths of a second.
- Calling that a force is awkward: the number is huge and it lasts no time at all.
- An impulse states the outcome instead: how much the motion changed. Its units are newton-seconds.

Three ways to deliver the same 4 N·s

$$\begin{array}{rclcl} 400 \text{ N} & \times & 0.01 \text{ s} & = & 4 \text{ N} \cdot \text{s} \\ 8 \text{ N} & \times & 0.50 \text{ s} & = & 4 \text{ N} \cdot \text{s} \\ 4 \text{ N} & \times & 1.00 \text{ s} & = & 4 \text{ N} \cdot \text{s} \end{array}$$

All three change a 0.5 kg ball's speed by exactly 8 m/s. Only the product matters — which is why you can hand the engine the bundle and forget about the timing.

Where impulses turn up

`ForceMode.Impulse`

Hand it a value in newton-seconds and it lands in one step. Heavier objects still respond less.

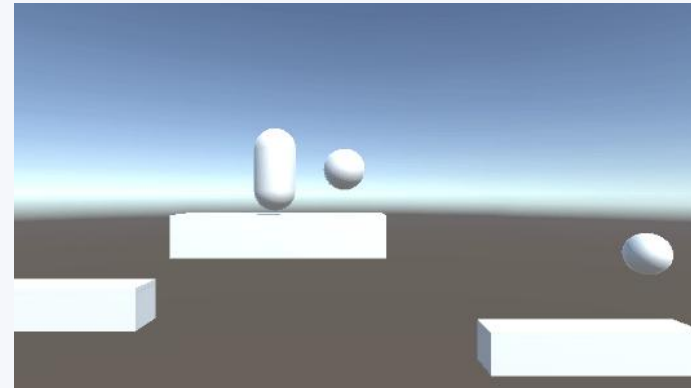
Every contact

Collision response is worked out as an impulse — which is why being hit is instant, not a gradual push.

`Collision.impulse`

Read it in `OnCollisionEnter` to find out how hard the hit was. Your damage and impact-sound input.

```
rb.AddForce(dir * 4f, ForceMode.Impulse);  
rb.AddForce(Vector3.up * 6.26f,  
            ForceMode.VelocityChange);
```



A WORKED EXAMPLE

Pick your numbers instead of guessing them

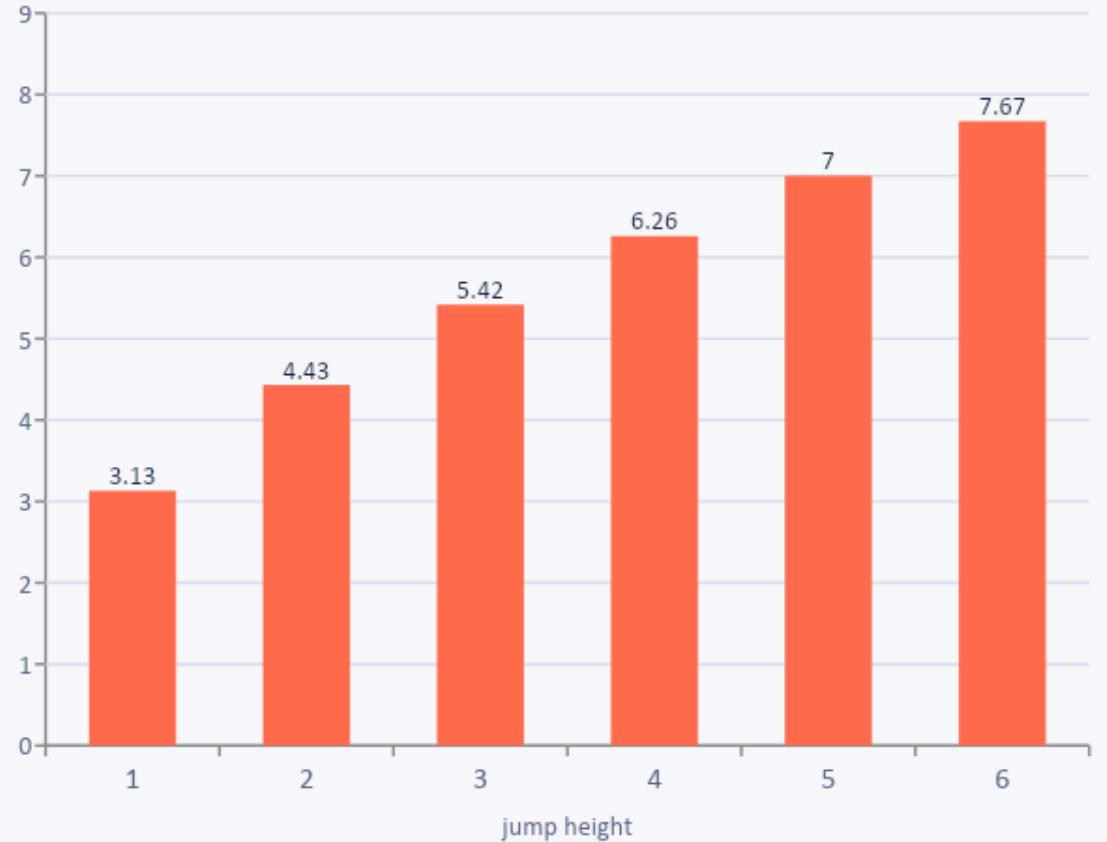
$$v_0 = \sqrt{2gh}$$

The commonest beginner loop is typing a force value, playing, and typing a bigger one. You rarely need to: if you know the jump height you want, the speed that gets you there is a one-line calculation.

A 2-metre jump, step by step

- $\sqrt{2 \times 9.81 \times 2} = 6.26$, so you need 6.26 m/s upward.
- Apply it with `ForceMode.VelocityChange` and every character reaches 2 m, whatever their mass.
- Gravity brings them back down on its own — you never push downward.
- Want a snappier feel? Raise gravity and recompute, rather than fighting it with extra force.

Upward speed needed for a given jump height



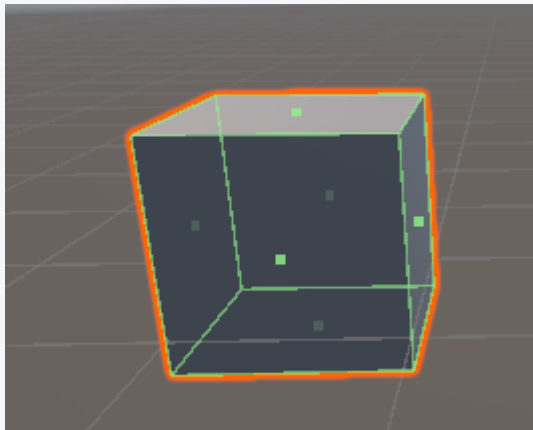
Doubling the height does not double the speed — it is a square root, so you only need about 1.4× as much.

A collider is a shape for physics only

1

It is invisible

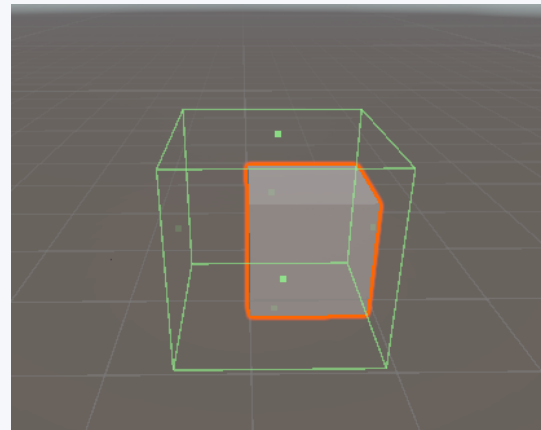
Colliders never render. In the editor you see them as green outlines; in the build they are pure maths.



2

It need not match the model

A rough approximation is faster and, in play, impossible to notice. Detail here buys you nothing.



3

It is what actually touches

Your beautiful mesh is decoration. Every contact in the game happens between colliders.

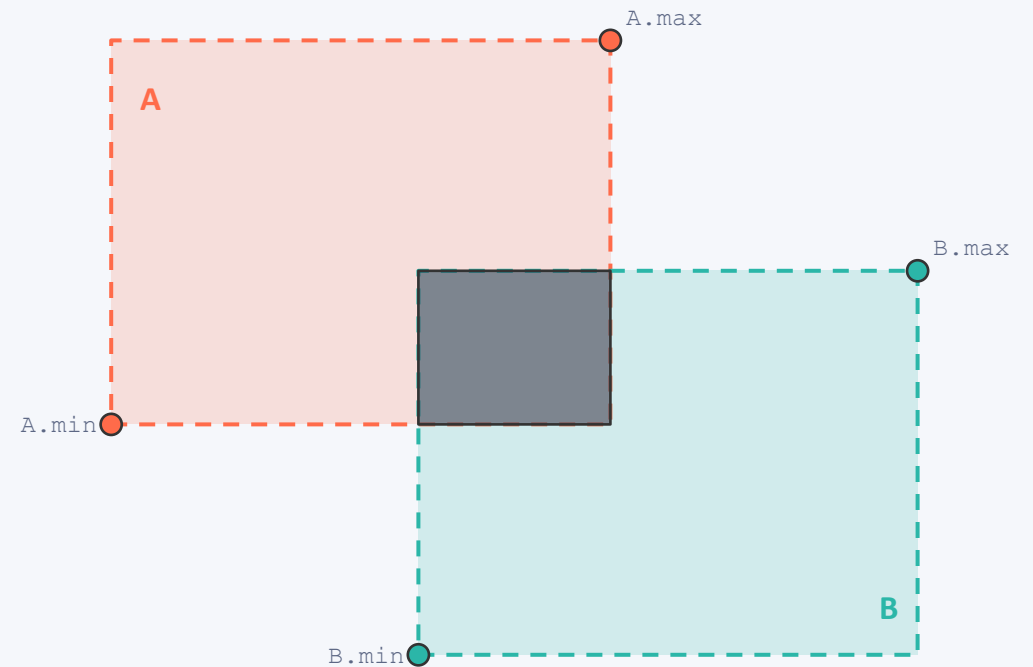
The box test underneath every collider

- Before any exact test, the engine wraps each collider in an axis-aligned bounding box — sides parallel to the world axes, stored as just a minimum and a maximum corner.
- No rotation to undo means no trigonometry: a handful of comparisons settles it.
- Only pairs whose boxes overlap go on to the real shape test.

```
// is a point inside the box?  
p.x >= min.x && p.x <= max.x &&  
p.y >= min.y && p.y <= max.y
```

```
// do two boxes overlap?  
a.min.x <= b.max.x && a.max.x >= b.min.x &&  
a.min.y <= b.max.y && a.max.y >= b.min.y
```

Two boxes, a handful of comparisons



Overlapping on both axes, so this pair is worth testing properly. Fail either one and the engine moves on.

Where you have already seen bounding boxes

Minecraft

Every block, player and mob is an axis-aligned box, and so is every chunk of the world. The whole game is boxes inside boxes — the next slide takes it apart. F3 + B draws them on screen.

2D platformers

Celeste, Super Meat Boy, Hollow Knight: the player is a box that never rotates. Overlap is four comparisons, so the movement code can be exact — which is what makes the controls feel so precise.

Every engine's first pass

Unity, Unreal, Godot and Box2D all wrap each collider in a box and compare those first. Your capsule character is tested as a box before it is tested as a capsule.

Rendering and ray tracing

Boxes decide what is on screen at all, and hardware ray tracing walks a tree of nested boxes to find which triangles a ray could possibly hit.

The catch: a bounding box cannot rotate. Turn the object and the box has to grow to contain it, so it stops being a good fit — which is exactly why boxes stay a first pass and the precise test uses the real shape.



Minecraft: boxes inside boxes

1 Blocks and entities

A player is a box 0.6 wide and 1.8 tall. To move, the game collects the block boxes lying inside the box it is about to occupy, and stops it at the first one.

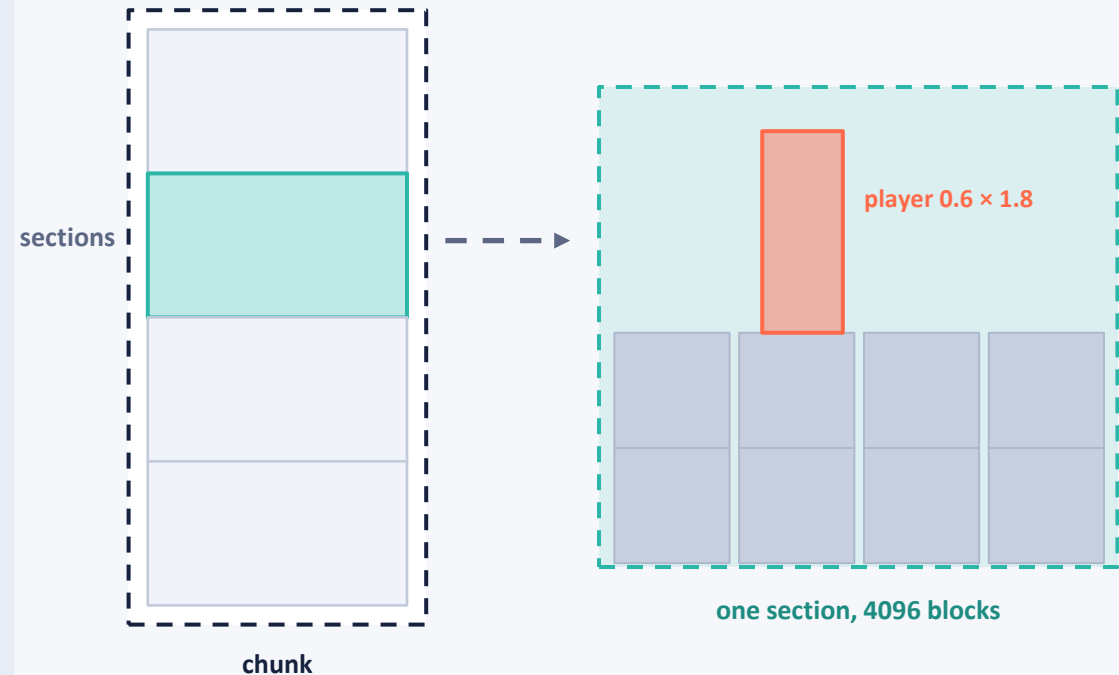
2 Chunk sections

Blocks are stored in $16 \times 16 \times 16$ sections, each with its own bounding box. If your box misses the section's box, all 4096 blocks inside are skipped without being looked at.

3 Chunks and the world

Sections stack into 16×16 chunk columns. Only the chunks near a player are loaded and ticked — the rest do not exist as far as physics is concerned.

The same trick at three scales



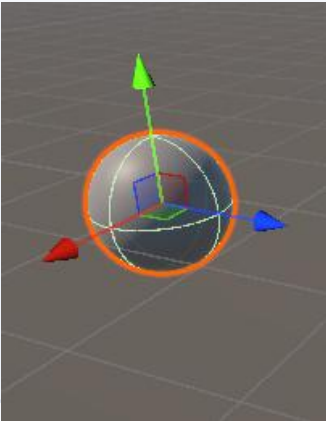
Test the cheap outer box first, and only look inside if it overlaps. That nesting is what a bounding volume hierarchy is — and it is why a world of billions of blocks stays cheap.

Four types, in order of preference

1

Sphere

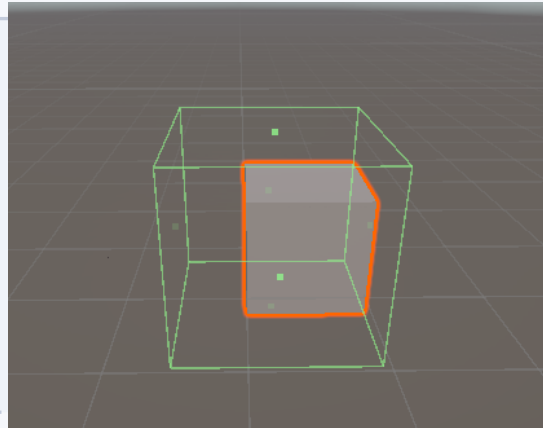
One distance against one radius — the cheapest test there is. Balls, pickups, blast radius.



2

Box

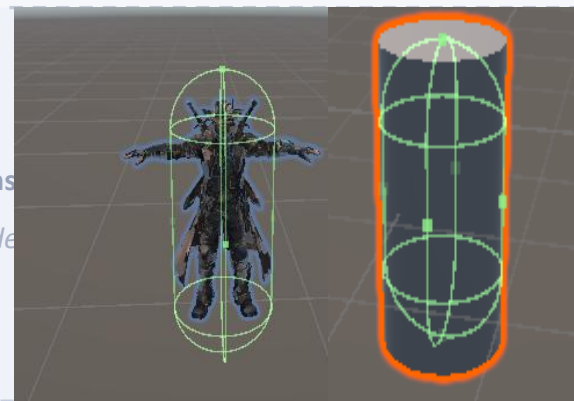
Walls, crates, platforms, floors. The default for anything with flat sides.



3

Capsule

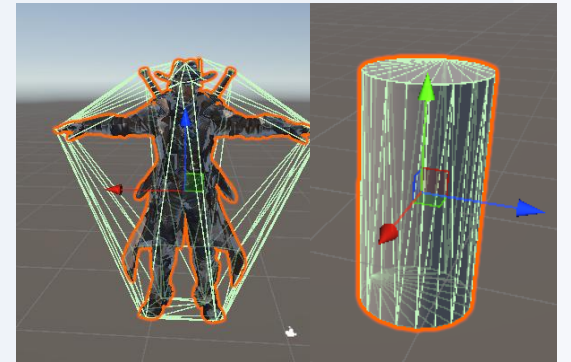
The standard for characters: rounded ends slide up slopes instead of catching on edges.



4

Mesh

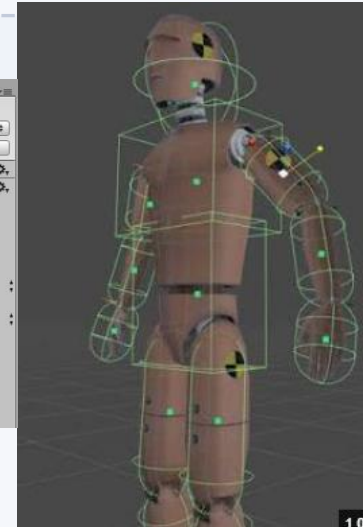
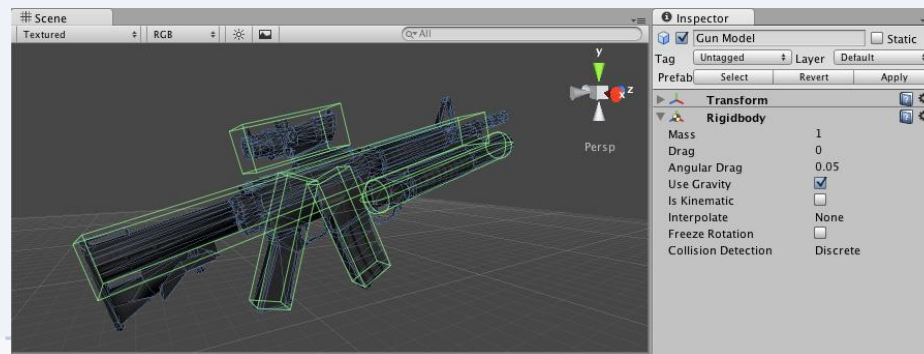
Follows the model exactly and costs accordingly. Do not use one unless you have to.



Compound colliders

You are not limited to one collider per object. Stack several primitives on the same object, or on child objects, and you can trace a surprisingly complicated shape — a chair, a lamp post, a vehicle — out of boxes and capsules.

- Almost always faster than a mesh collider.
- Works on dynamic objects, where a concave mesh collider cannot.
- Nobody playing will ever tell the difference.



What happens the moment two objects touch

1

It notices the overlap

The engine checks positions 50 times a second. If two colliders have started to overlap, it records where they touched and which way the surface faces — the contact normal.

2

It pushes them apart

Rather than a force, the engine applies an instant kick along that normal, just big enough that the two are no longer moving into each other. That is why contact feels immediate.

3

It shares the kick out by mass

Both objects get the same size of kick in opposite directions, so the lighter one changes speed far more. Against a wall or floor, all of it lands on you.

Heavy meets light

A lorry hitting a football gives both the same kick. The football rockets away; the lorry barely notices. This is why a very heavy object resting on a very light one looks broken — keep interacting masses within roughly ten times of each other.

Fast objects can miss entirely

Because positions are only checked 50 times a second, an object moving at 100 m/s jumps 2 m between checks and can be in front of a wall in one check and behind it in the next. Set Collision Detection to Continuous on bullets and fast vehicles — and only on those, since it costs more.

Step one: find the contact

1

What the engine looks for

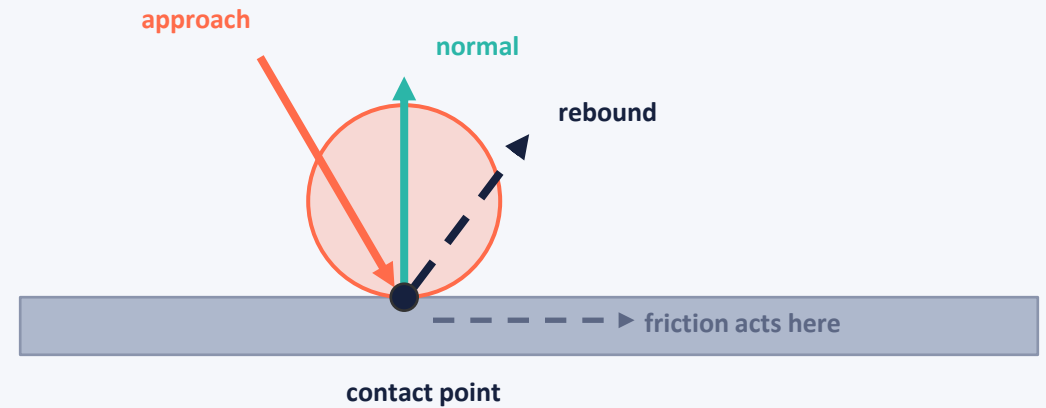
The exact point where the shapes meet, the normal — the direction perpendicular to the surface — and how deeply they have overlapped.

2

Then it measures the approach

Only the part of the relative velocity running along the normal counts. If the two are already moving apart, there is nothing to do.

One contact, drawn out



Step two: work out the response

3

Work out one number

The impulse that would exactly cancel the approach, scaled by bounciness and shared out by the two masses. An immovable object counts as infinite mass.

4

Apply it both ways

Equal and opposite along the normal. Each object's speed changes by that impulse divided by its own mass — which is why the lighter one flies.

5

Friction, then separation

A second, smaller impulse along the surface, capped by the friction values. Any leftover overlap is eased out over the next few steps.

$$j = \frac{-(1+e)(\vec{v}_{rel} \cdot \hat{n})}{\frac{1}{m_1} + \frac{1}{m_2}}$$

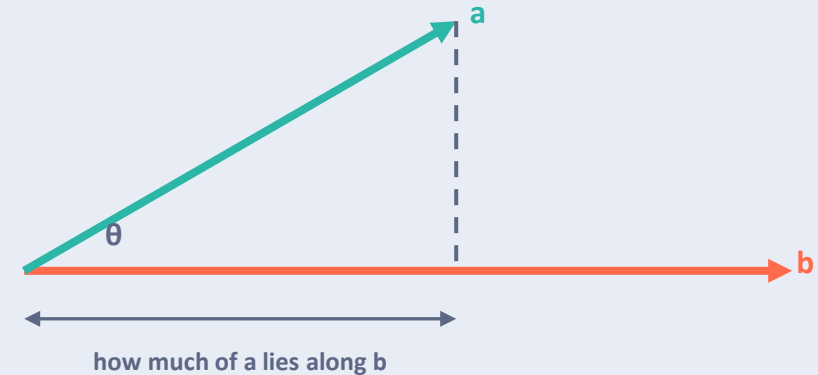
e is the bounciness and the top line is the approach speed along the normal — cancel the approach, add back as much as the bounciness allows, and let the masses decide who moves.

The dot product: are we pointing the same way?

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z = |\vec{a}| |\vec{b}| \cos \theta$$

- It returns a single number, not a vector.
- With unit vectors it is exactly $\cos \theta$, so the answer lands between -1 and 1.
- The sign alone is useful: positive means roughly the same direction, negative means roughly opposite.
- $|a| \cos \theta$ is the length of a's shadow on b — how much of a points along b.

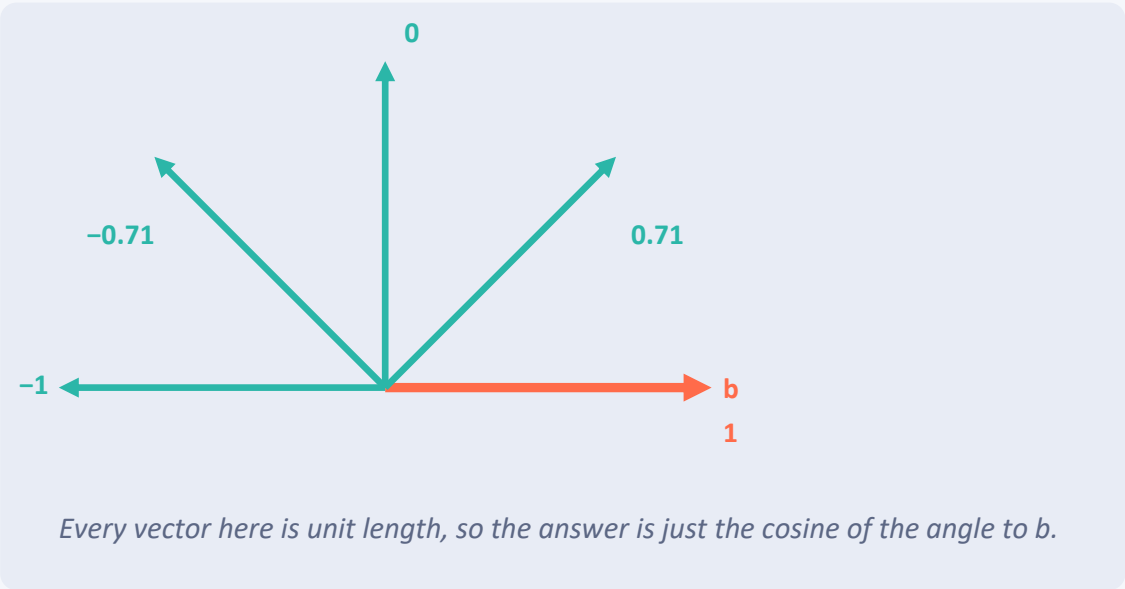
1 same · 0 perpendicular · -1 opposite



```
Vector3.Dot(transform.forward, toTarget) > 0      // is it in front of me?
Vector3.Dot(hit.normal, Vector3.up) > 0.7f        // floor, or wall?
Vector3.Dot(c.relativeVelocity, normal)           // the approach speed from earlier
```

Reading the dot product at a glance

$\hat{a} \cdot \hat{b} = \cos \theta$



a · b	angle	what it tells you
1.0	0°	pointing exactly the same way
0.7	45°	broadly the same way — within a wide cone
0.0	90°	at right angles; neither with you nor against
-0.7	135°	broadly opposite
-1.0	180°	exactly opposite — head-on

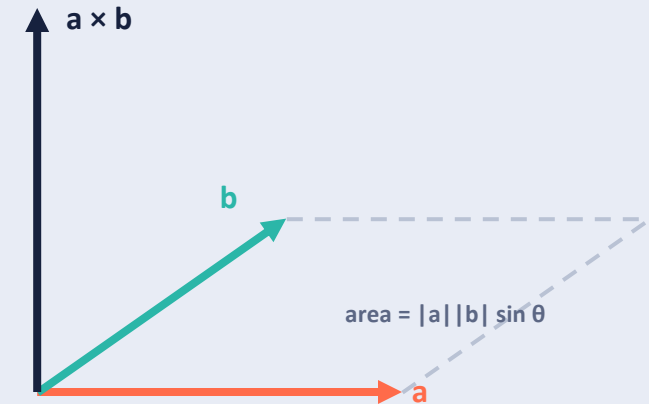
The point: you almost never need the angle itself — the sign, or a threshold, is enough.

```
if (Vector3.Dot(a, b) > 0f)           // less than 90° apart, and no acos needed
float sqr = Vector3.Dot(v, v);       // squared length — compare distances cheaply
if (Dot(fwd, dir) > Mathf.Cos(fov * 0.5f * Mathf.Deg2Rad)) // inside the view cone
```

The cross product: a vector at right angles to both

$$|\vec{a} \times \vec{b}| = |\vec{a}| |\vec{b}| \sin \theta \quad \vec{a} \times \vec{b} \perp \vec{a}, \vec{b}$$

- It returns a vector, not a number — one that sticks straight out of the plane the two vectors span.
- Its length is the area of the parallelogram they make, so it is zero when they are parallel.
- Order matters: $\vec{a} \times \vec{b}$ points the opposite way to $\vec{b} \times \vec{a}$.
- In Unity's left-handed system the direction follows the left-hand rule: fingers along $\vec{a}$, curl to $\vec{b}$, thumb gives the answer.



```
Vector3.Cross(edge1, edge2).normalized // the normal of a triangle face
Vector3.Cross(transform.up, forward)    // a vector pointing to the side
Vector3.Cross(forward, toTarget).y > 0  // is the target left or right of me?
```

Reading the cross product at a glance

$$|\hat{a} \times \hat{b}| = \sin \theta$$

nearly parallel → almost nothing



$$|\mathbf{a} \times \mathbf{b}| \approx 0.15$$

square on → the maximum



$$|\mathbf{a} \times \mathbf{b}| = 1$$

Mirror image of the dot product: the dot peaks when two vectors line up, the cross peaks when they are at right angles — and it collapses to zero when they are parallel.

$ \mathbf{a} \times \mathbf{b} $	angle	what it tells you
0	0° or 180°	parallel — same line, no plane between them
0.7	45°	leaning together, but not aligned
1.0	90°	square on — the largest area they can span
sign	—	which side: swap the order and it flips

Use them together: the dot says how much two directions agree, the cross says which side. Between them you have the full bearing.

```
if (Vector3.Cross(a, b).sqrMagnitude < 0.0001f) // parallel — no plane, no normal
float side = Vector3.Cross(transform.forward, toTarget).y;
// side > 0 the target is to your right, side < 0 to your left
```

Is it in front of me, or behind?

$$\hat{f} \cdot (\vec{p}_{target} - \vec{p}_{self}) > 0$$

1 Take the direction to it

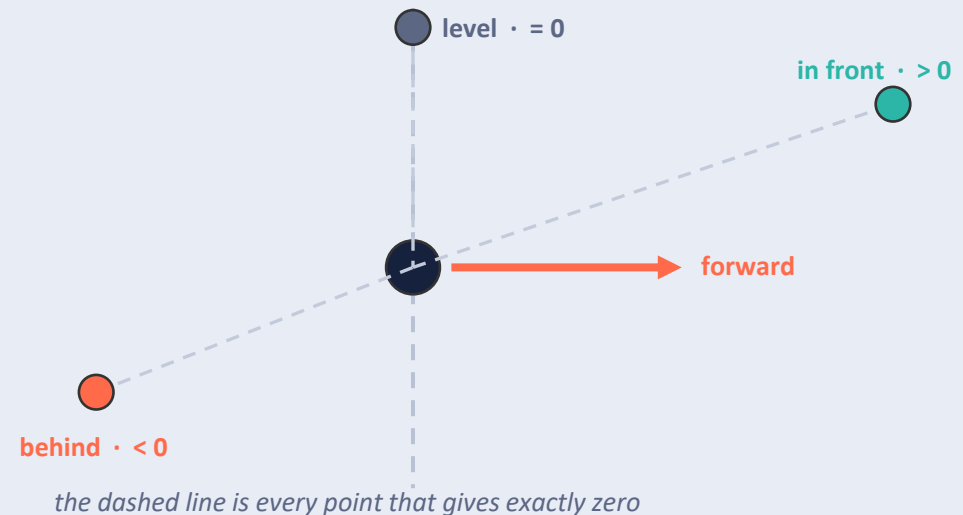
`target.position - transform.position`. Not normalised — for a sign test you do not need to be.

2 Dot it with your forward

`transform.forward` is already a unit vector pointing out of the object's nose.

3 Read the sign

Positive is in front, negative is behind, zero is exactly level with you. One line, no angles.

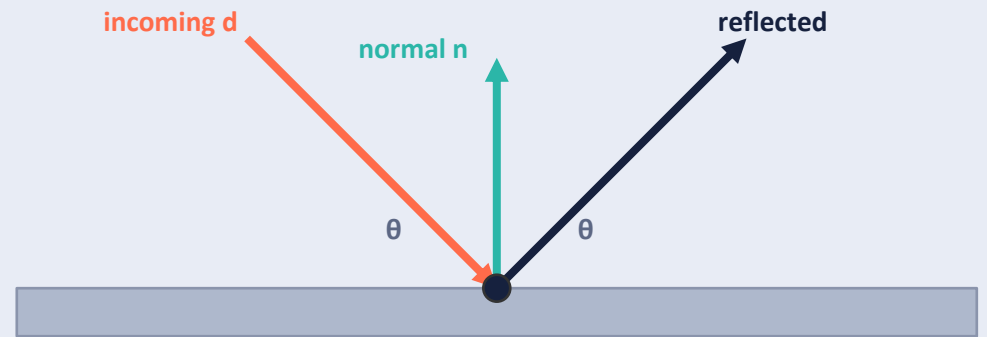


```
Vector3 to = target.position - transform.position;  
if (Vector3.Dot(transform.forward, to) > 0f) Ahead(); else Behind();  
if (Vector3.Dot(transform.forward, to.normalized) > 0.5f) InMy120DegreeCone();
```

Reflection: bouncing a direction off a surface

$$\vec{r} = \vec{d} - 2(\vec{d} \cdot \hat{n})\hat{n}$$

- Split the incoming direction into the part along the normal and the part along the surface.
- Flip the normal part, keep the other, and you have the bounce — angle in equals angle out.
- The dot product is doing the splitting, which is why it appears in the middle of the formula.
- $\hat{n}$ must be a unit vector. `hit.normal` already is.



```
Vector3 bounce = Vector3.Reflect(direction, hit.normal);  
// ricochets, laser bounces, deflected projectiles — and it is exactly what a  
// perfectly bouncy contact does to the part of the velocity along the normal
```

Bounciness and friction

Bounciness

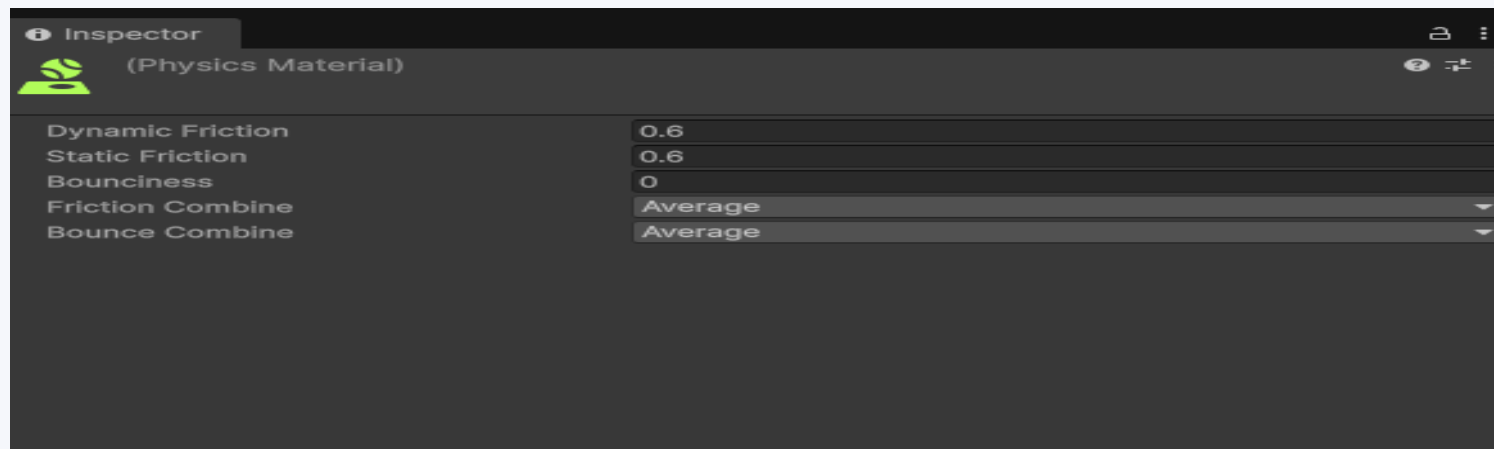
0 = dead thud 1 = never stops

- At 0.5 a ball returns to a quarter of the height you dropped it from.
- 0.8 is already very lively. Above 1 it gains energy every hit and the sim breaks.

Friction

0 = ice 1 = rubber on tarmac

- Grip depends on how hard the surfaces are pressed together, so a heavier crate is harder to shove.
- Static friction must be beaten to start moving; dynamic friction applies once sliding.



Which value wins?

Every contact has two materials and has to end up with one number. The Combine setting decides how — and the four options are ranked, with the higher-ranked one winning for that contact.

Average

Minimum

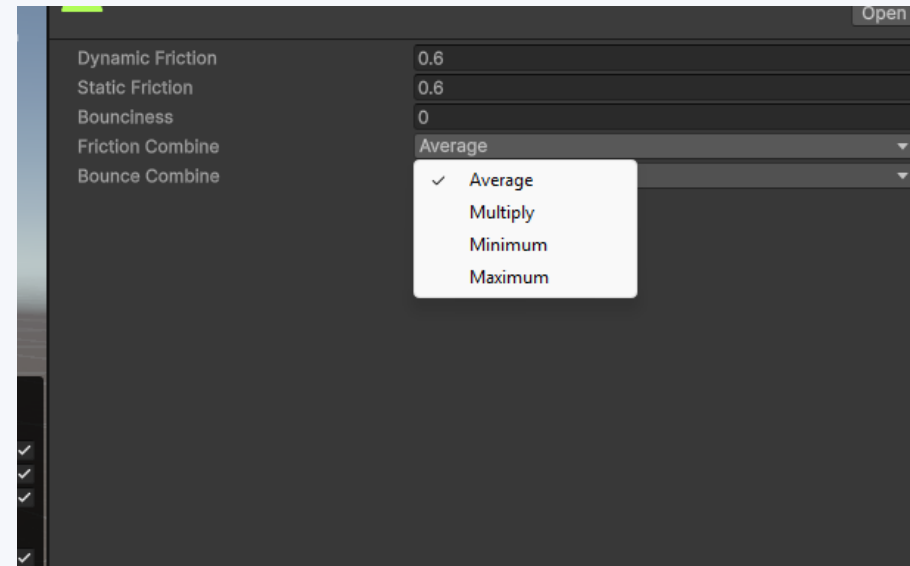
Multiply

Maximum

lowest priority → highest priority

What goes wrong

One prop set to Maximum bounciness overrides every surface it touches, however carefully the others were tuned. That is usually the culprit when a whole level starts to feel springy.



A trigger is a collider with its solidity switched off

Same component, two jobs

There is no Trigger component. You add a Box, Sphere or Capsule Collider exactly as before, then tick Is Trigger on it. The shape is unchanged and the engine still tracks every overlap — it simply stops resolving them, so nothing is pushed apart and nothing is stopped.

Three things people forget

- A trigger still needs a Rigidbody somewhere in the pair — two static objects overlapping report nothing at all.
- If the trigger itself has to move, give it a kinematic Rigidbody so its overlaps keep updating as it goes.
- It hands you a Collider, not a Collision: no contact point, no normal, no impulse, because nothing was resolved.

Is Trigger off — solid



stopped, then bounced back — `OnCollisionEnter` fires

Is Trigger on — a volume



it carries straight on — only the event is raised

Nothing firing? Work down this list

1

Is a Rigidbody involved?

At least one of the two objects needs one, or the pair is never even tested.

2

Do the layers talk?

Check the Layer Collision Matrix. An unticked box silently disables the pair.

3

Right method, right kind?

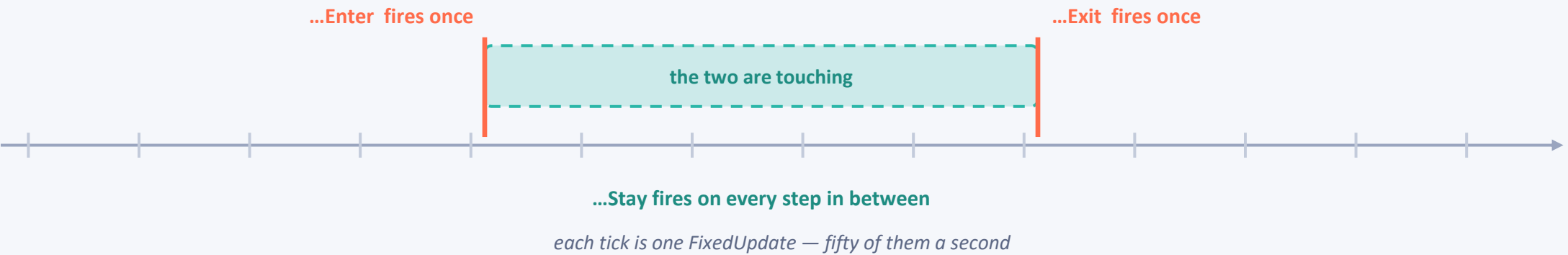
A trigger fires OnTrigger..., a solid contact fires OnCollision... Mixing them up is the usual cause.

4

Is the script in the right place?

It has to be on the object that owns the collider, not on a parent that has none.

Six methods, and when each one fires



Method	You receive	When it fires
OnCollisionEnter	Collision	The first step in which the two solid colliders are touching.
OnCollisionStay	Collision	Every step for as long as they stay in contact — keep the body of this one cheap.
OnCollisionExit	Collision	The step in which they stop touching.
OnTriggerEnter	Collider	The first step in which something enters the trigger volume.
OnTriggerStay	Collider	Every step while it is still inside.
OnTriggerExit	Collider	The step in which it leaves — and also if it is disabled or destroyed while inside.

Reading a solid contact

```
void OnCollisionEnter(Collision c)
{
    ContactPoint p = c.GetContact(0);
    Spark(p.point, p.normal);

    if (c.impulse.magnitude > 5f)
        PlayCrashSound();
}
```

Screenshot

e.g. the Console output, or the impact effect in play mode

What the Collision object hands you

gameObject

What you hit — tag it, damage it, look up a component on it.

GetContact(i)

The point and the normal; contactCount says how many there were.

relativeVelocity

How fast the two were closing just before the hit.

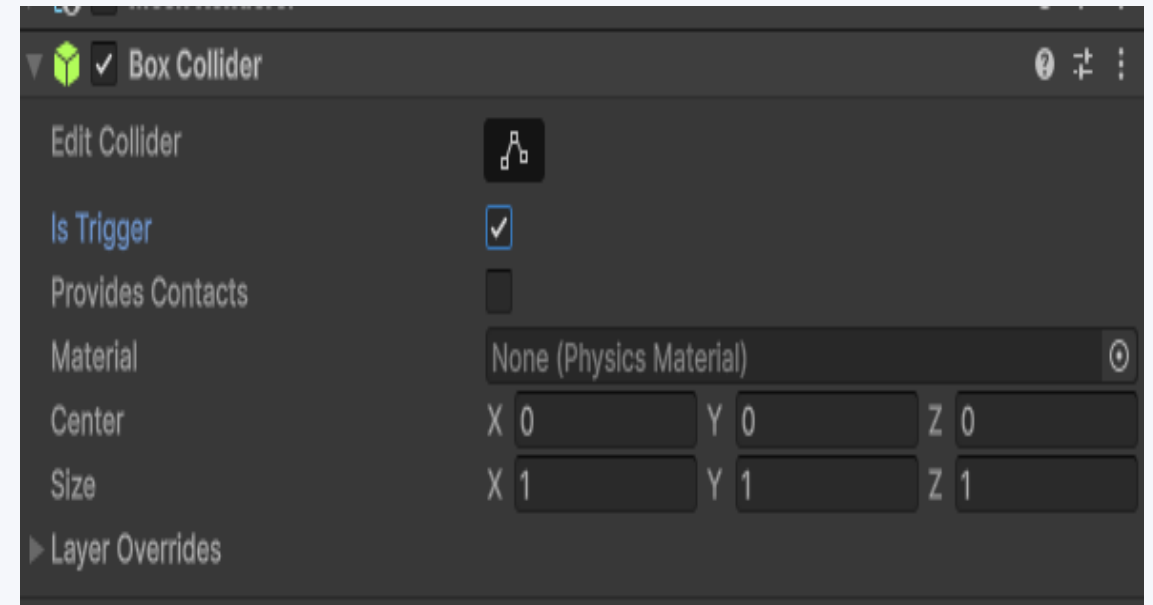
impulse

How hard the engine had to push to separate them.

Reading a trigger

```
void OnTriggerEnter(Collider other)
{
    if (other.CompareTag("Player"))
    {
        Collect();
        Destroy(gameObject);
    }
}
```

- You are handed a Collider, not a Collision — no point, no normal, no impulse, because nothing was resolved.
- CompareTag is the cheap way to ask what walked in.
- The script goes on the object with the trigger, not on a parent.



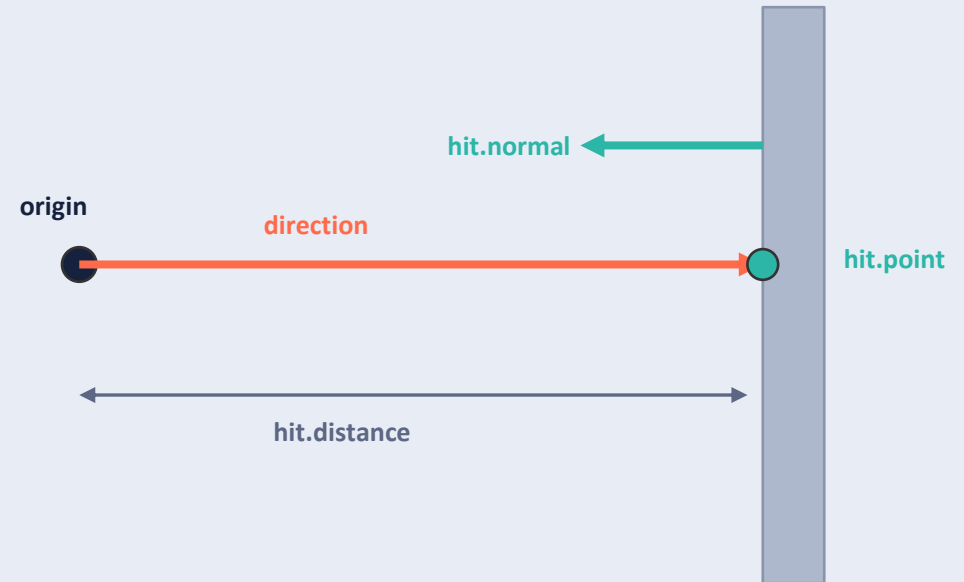
Physic cast (Raycasting) : Asking the engine instead of waiting for it

The six collision callbacks only tell you about things that have already happened.

When you need an answer right now — is there ground under my feet, what is in front of the gun, who is inside the blast — you ask the physics world directly instead.

```
if (Physics.Raycast(transform.position,  
    Vector3.down, out RaycastHit hit,  
    1.1f, groundMask))  
{  
    grounded = true;  
}
```

A ray, and what comes back



hit.collider tells you what you hit.

Three ways to ask

1

Raycast

Fires a line and returns the first thing it meets: point, normal, distance, collider. Ground checks, aiming, hitscan weapons, line of sight.

2

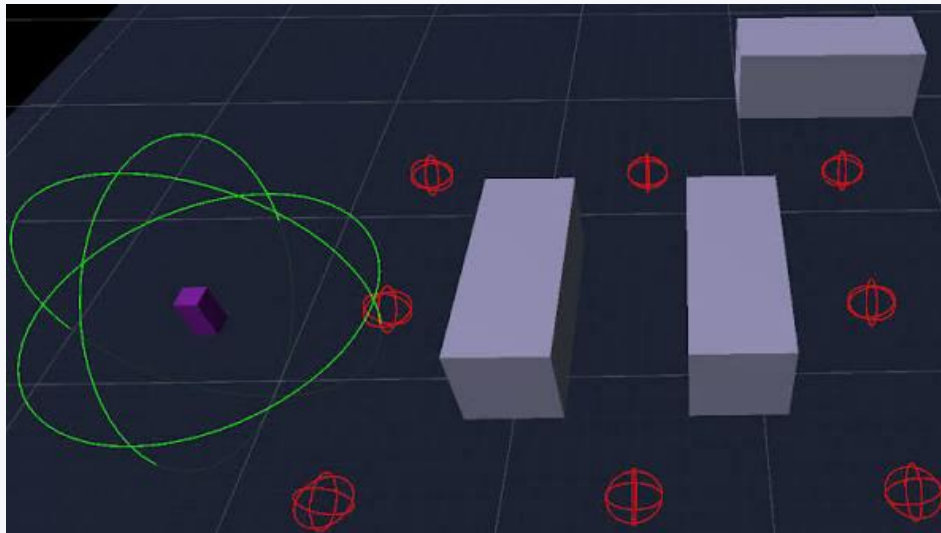
SphereCast CapsuleCast

The same sweep with thickness, so it cannot slip through a narrow gap. Much closer to how a character actually moves.

3

OverlapSphere OverlapBox

Everything inside a region at this instant, as an array. Explosions, proximity checks, area effects.



Layers: deciding what is even tested

Every GameObject sits on exactly one of 32 layers — eight are Unity's, the rest are yours to name.

The Layer Collision Matrix

Project Settings › Physics. Untick a box and that pair of layers is never tested — thrown out before any shape maths runs, which makes it the cheapest optimisation in the engine.

A sensible starting set

Player, Enemy, PlayerBullet, EnemyBullet, Ground, Interactable. Bullets rarely need to test against other bullets, and enemy fire should ignore enemies.

Layer Collision Matrix

Ticked pairs are tested. The rest never meet.

	Player	Enemy	Bullet	Ground
Player	☐	☒	☒	☒
Enemy	☒	☒	☒	☒
Bullet	☒	☒	☐	☒
Ground	☒	☒	☒	☐

Bullets ignore bullets, level geometry never tests itself, and there is only one player — three whole classes of pair the engine can forget about.

The same idea in code: LayerMask

```
int ground = LayerMask.GetMask("Ground",  
                                "Platform");  
  
Physics.Raycast(pos, Vector3.down,  
                out hit, 1.1f, ground);
```

- A query only sees the layers you let through, so a ground check cannot be fooled by an enemy walking past.
- The mask is a bitfield, not a layer number. The classic bug is passing the index straight in.
- To build one by hand: `1 << gameObject.layer`.

Handling a squeeze the engine cannot

1

Crush it

The standard game answer. Detect the squeeze and destroy or damage the trapped object rather than asking physics to resolve it.

2

Sweep before you move

Cast the platform's shape along the motion it is about to make. If something is in the way, stop it, reverse it, or crush what it found.

3

Leave an escape route

Physics can solve a squeeze if there is somewhere to go. A gap above or to the side turns an impossible contact into an ordinary push.

4

Never teleport into a body

Assigning a kinematic transform inside a dynamic object creates a deep overlap, and the separation impulse launches it. MovePosition sweeps; assignment does not.

```
Vector3 step = dir * speed * Time.fixedDeltaTime;

if (rb.SweepTest(dir, out RaycastHit hit, step.magnitude))
    Blocked(hit);    // stop, reverse, or crush
else
    rb.MovePosition(rb.position + step);
```

One unit is one metre

The contract

- Metres, kilograms and seconds throughout.
- A 1×1×1 cube is one metre across.
- Force in newtons, impulse in newton-seconds.

Why “floaty” is usually a scale bug

Import a character at ten times its real size and it is 18 metres tall. Falling its own height now takes over three times as long. The physics is right — the object is enormous. Fix the model, not the gravity.



Two things that look like bugs and are not

Jitter

Contacts are resolved once per fixed step, in pairs. A moving object can be clear of a wall on one step and overlapping it on the next; the engine pushes it back out, sometimes a little too far, and the next step it falls back in. Repeat fifty times a second and you get a visibly buzzing object.

What to try

- Set Collision Detection to Continuous on the object that is moving fast.
- Turn on Interpolate — it smooths what you see between steps without changing the simulation.
- Make sure nothing starts the scene already overlapping something else.

Sleeping

Once a Rigidbody drops below a minimum speed, the engine decides it has come to rest and stops updating it — it goes to sleep. No processor time is spent on a sleeping body at all, which is what makes a scene full of settled crates cheap.

What to watch for

- A sleeping body wakes on a collision or a force, so most of the time you never notice.
- But if an object ignores something you changed in code, check whether it is simply asleep.
- `rb.WakeUp()` forces the issue; so does any `AddForce`.

The pattern: both come from the same source — the simulation only exists at fifty discrete moments a second, and anything that happens between two of them has to be inferred. Most physics oddities are a version of this.

Every equation in this deck, in one place

Newton's second law

the whole engine, in one line

$$F = m a \quad \implies \quad a = \frac{F}{m}$$

The net force this step

what actually moves the object

$$\sum \vec{F} = \vec{F}_1 + \vec{F}_2 + \vec{F}_3 + \dots \quad \vec{a} = \frac{\sum \vec{F}}{m}$$

One integration step

how velocity and position are updated

$$\vec{v} \leftarrow \vec{v} + \frac{\sum \vec{F}}{m} \Delta t \quad \vec{x} \leftarrow \vec{x} + \vec{v} \Delta t$$

Dot product

how much of one vector lies along another

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z = |\vec{a}| |\vec{b}| \cos \theta$$

Cross product

a vector at right angles to both

$$|\vec{a} \times \vec{b}| = |\vec{a}| |\vec{b}| \sin \theta \quad \vec{a} \times \vec{b} \perp \vec{a}, \vec{b}$$

Every one of these is doing the same job: turning what is happening now into what should happen next.

Six habits that prevent most physics bugs

1 Put physics code in FixedUpdate

Read input in Update, act on it in FixedUpdate. Forces applied in Update behave differently on every machine.

2 Never set transform.position on a Rigidbody

That teleports the object past every check. Use `rb.MovePosition`, or set its velocity and let physics carry it.

3 Model at real-world size

One unit is one metre. Almost every complaint that physics feels floaty is really a model that is ten times too big.

4 Keep masses within about 10×

Wildly different masses in contact are the main cause of sinking, jittering and exploding stacks.

5 Prefer box, sphere and capsule colliders

They are far cheaper and better behaved than a mesh collider, and players never notice the difference.

6 Turn on Continuous only for fast objects

Bullets and vehicles yes, crates and debris no — it costs real performance per object.

Forces change how things move. Contacts stop them moving through each other. Everything else is detail.

Further reading: docs.unity3d.com/Manual/PhysicsOverview.html